WEEK 3 (Praveen 2520030406)

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        printf("fork() failed\n");
        exit(1);
    }

    else if (pid == 0)
    {
        // Child Process
        printf("\n--- Child Process ---\n");
        printf("Child PID: %d\n", getpid());
        printf("Child PPID: %d\n", getppid());

        printf("Child is running...\n");

        sleep(5);

        printf("Child is terminating...\n");
        exit(0);
    }

    else
    {
        // Parent Process
        printf("\n--- Parent Process ---\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child terminated.\n");
        printf("Parent process terminating...\n");
    }

    return 0;
}
```

Output:

```
[praveenbusala@Praveens-MacBook-Air-2 CLG % nano week3.c
[praveenbusala@Praveens-MacBook-Air-2 CLG % gcc week3.c
[praveenbusala@Praveens-MacBook-Air-2 CLG % ./a.out
 Parent Process Started
 Parent PID: 24185

--- Parent Process ---
Parent PID: 24185
Child PID: 24186
Parent is waiting for child...

--- Child Process ---
Child PID: 24186
Child PPID: 24185
Child is running...
Child is terminating...
Child terminated.
Parent process terminating...
praveenbusala@Praveens-MacBook-Air-2 CLG %
```